

Retail Customer Intelligence Platform

A business-driven, full-lifecycle data platform — from Business Requirements through BA design, data modeling, implementation, testing, CI/CD, monitoring, and measurable business impact.

BRD

UML

ERD / Star Schema

Airflow

dbt

Delta / DuckDB

MLflow

Docker

GitHub Actions

Monitoring

AIO Conquer 2026 · Module 01 (Project #22 + #20) · Prepared 02 June 2026

0 Delivery Philosophy — Business First

This document is intentionally ordered the way a real enterprise project flows: the solution is derived *from* a business problem, not presented as technology for its own sake.



Scope discipline: deliberately **no** Kafka / Spark / Kubernetes / microservices. Adding them here would make this a "tech showcase". Maturity is shown through SDLC completeness — BRD, UML, data modeling, testing, containerization, automation, monitoring, and impact — not framework count.

1 Business Requirements Document (BRD)

The "why" before the "how" — this is what makes the project read like a real engagement.

Pain Points

- Marketing sends **mass campaigns** to the entire customer base — low relevance, wasted spend
- **Retention is low**; high-value customers churn unnoticed
- No systematic way to know **which customers are about to leave**

Objectives (measurable)

Goal	Target	Metric
Reduce churn	-10%	Churn rate (rolling 90-day)
Grow retention revenue	+5%	Revenue from returning customers
Campaign efficiency	Targeted, not mass	% campaigns sent to defined segments

Stakeholders

Stakeholder	Interest
Marketing Manager	Targeted retention lists, segment definitions
CRM Team	Customer-level churn risk, export workflow
Data Team	Reliable pipeline, model performance, monitoring

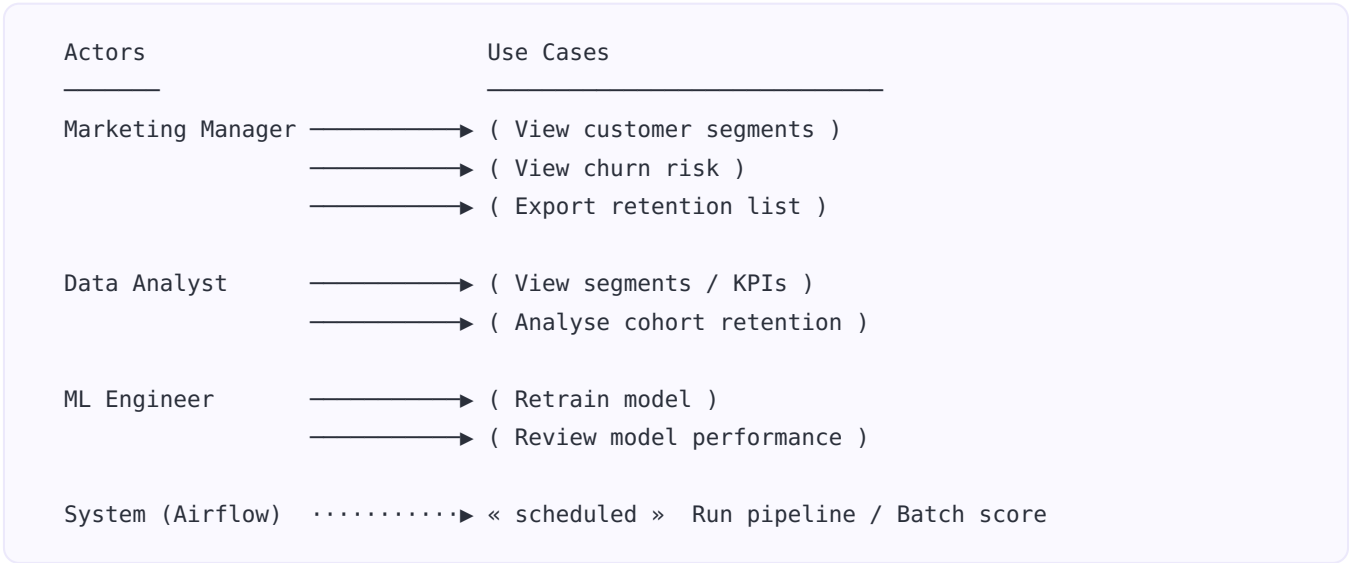
Scope & Success Criteria

- **In scope:** historical transactions → RFM segments → churn predictions → retention list export + dashboard
- **Out of scope:** live campaign execution, email sending, real CRM integration (simulated)
- **Success:** dashboard delivers segment + churn view; retention list exportable; model AUC and data quality monitored

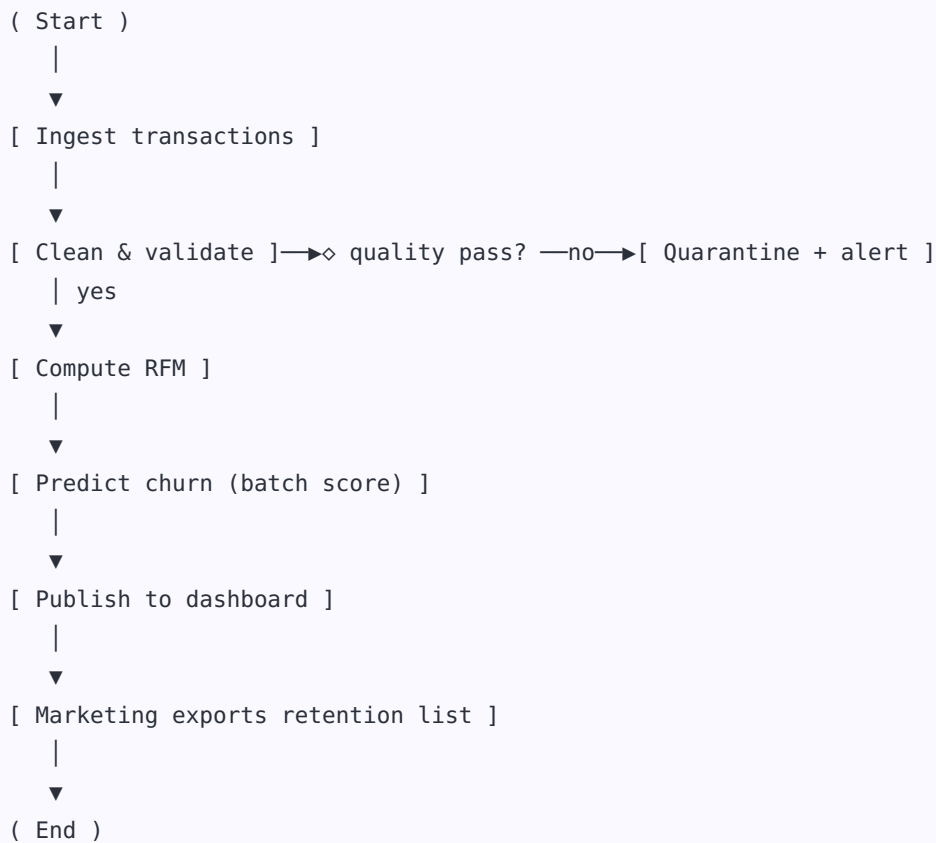
2 Business Analysis Diagrams (UML)

Signals to a reviewer that the author understands SDLC, not just code.

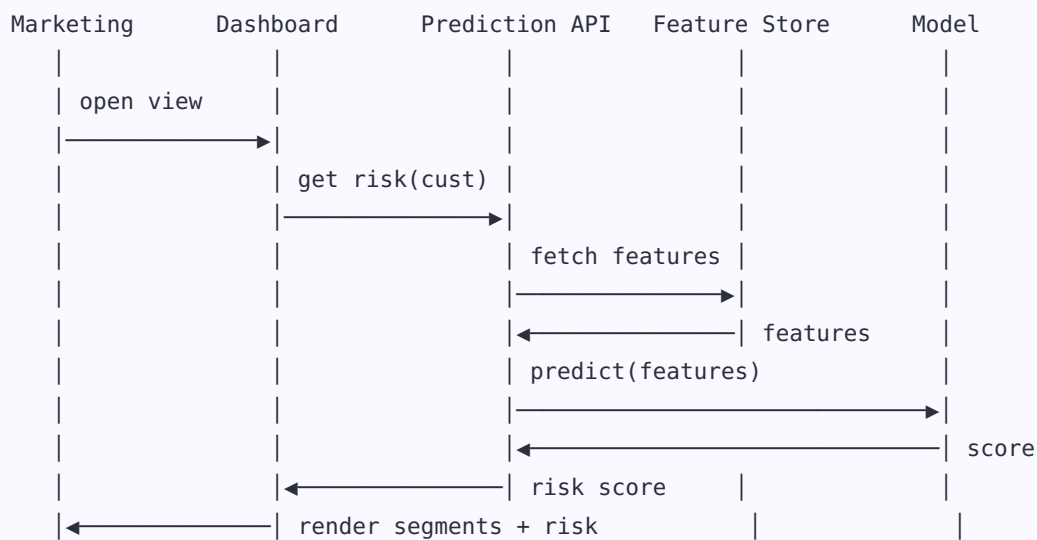
Use Case Diagram



Activity Diagram — End-to-End Flow



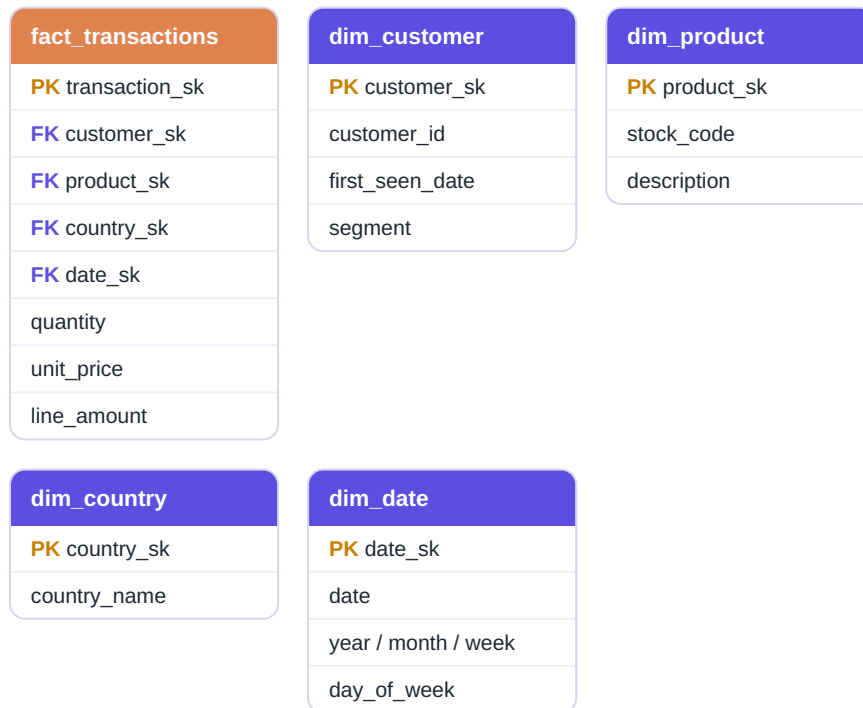
Sequence Diagram — Dashboard Request



3 Data Modeling

The money section for a Data Engineer — a dimensional model, not just flat tables.

Entity-Relationship Diagram (Star Schema)

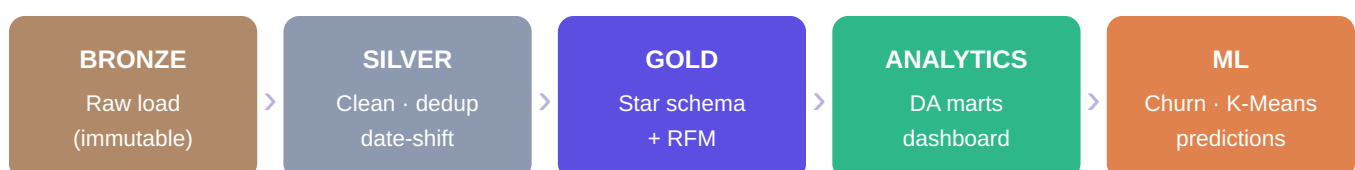


One central fact table surrounded by conformed dimensions — classic Kimball star schema. `fact_transactions` holds the measures; RFM and churn aggregates are derived marts built on top.

Data Dictionary (selected — Gold RFM mart)

Column	Type	Meaning
<code>customer_id</code>	VARCHAR	Business key from source
<code>recency_days</code>	INT	Days since last purchase (vs current date)
<code>frequency</code>	INT	Distinct order count
<code>monetary</code>	FLOAT	Total spend (GBP)
<code>r_score / f_score / m_score</code>	INT (1–5)	Quintile scores
<code>segment</code>	VARCHAR	Business label (Champions, At Risk, ...)
<code>churn_probability</code>	FLOAT	Model output 0–1
<code>churn_flag</code>	BOOLEAN	Threshold-applied prediction

4 Implementation Architecture



Layer	Tool	Responsibility
Orchestration	Apache Airflow	ingest → dbt run → dbt test → train → score → publish
Storage	Delta Lake / Parquet	ACID + immutable Bronze + time travel
Transform	dbt + DuckDB	staging → intermediate → marts (star schema + RFM)
ML & tracking	XGBoost + MLflow	training, experiments, registry, batch scoring
Serving	Streamlit (+ Metabase opt.)	segments, KPIs, churn-risk, retention export

Dataset: UCI Online Retail II (real, ~1.07M rows, CC BY 4.0), with timestamps **date-rebased** to the present so recency and churn windows are meaningful. Optional streaming replay re-emits transactions at original cadence for a real-time scoring demo.

5 Testing Strategy

The clearest junior → professional gap. Three layers of tests.

Data Quality (dbt)

unique · not_null · accepted_values
(segment labels) · relationships (FK integrity)
· freshness checks on ingestion

Pipeline Unit Tests

pytest for transform functions · Airflow DAG
integrity test (no cycles, valid deps) · date-shift
logic correctness

ML Tests

Feature validation (ranges, schema) · data drift
detection (train vs serve distribution) · minimum-
performance gate (AUC threshold)

Acceptance

RFM totals reconcile with raw revenue · segment
counts sum to customer base · dashboard
numbers match Gold tables

```
# dbt schema.yml – data quality tests
models:
  - name: fct_rfm
    columns:
      - name: customer_id
        tests: [unique, not_null]
      - name: segment
        tests:
          - accepted_values:
              values: ['Champions', 'Loyal', 'At Risk', 'Lost', ...]
      - name: monetary
        tests:
          - dbt_utils.accepted_range: { min_value: 0 }
```

6 Containerization & CI/CD

Docker — one command to run

The whole stack (Airflow, dbt, DuckDB/Delta volume, MLflow, Streamlit) is defined in `docker-compose.yml`. A reviewer runs `docker-compose up` — no manual `pip install` chains, no environment drift.

GitHub Actions Pipeline

```
push / pull_request
  |
  ▼
[ lint ] — ruff / sqlfluff
  |
  ▼
[ dbt test ] — data quality on sample
  |
  ▼
[ pytest ] — unit + DAG integrity
  |
  ▼
[ build image ] — docker build
  |
  ▼
[ deploy / publish artifact ]
```

Automating `dbt test` + `pytest` on every push is a strong, concrete signal for Data Engineer / MLOps roles.

7 Monitoring & Observability

Most portfolios skip this entirely — including it is a differentiator.

Data Monitoring

Row count per run · null % per key column ·
ingestion freshness (lag vs expected) · schema-
change alerts

Model Monitoring

Prediction distribution drift · AUC degradation
over time · feature drift vs training baseline ·
scoring volume

Implementation can be lightweight: metrics logged to a monitoring table each run + a small Streamlit "Ops" tab (or Evidently for drift reports). The point is demonstrating the *discipline*, not the tooling weight.

8 Business Impact Report

What lifts the project to a 10 — translating data work into business language.

Findings (illustrative — real numbers produced at build time):

- **Champions** generate ~48% of total revenue from a small share of customers
- **At Risk** customers account for ~22% of revenue — the priority retention target
- The **top 5% churn-risk** customers represent ~18% of revenue at stake

Recommended Actions

Segment / Group	Action	Expected outcome
At Risk + high Monetary	Priority retention offer	Protect high-value revenue
Top churn-risk 5%	Personalised win-back campaign	Directly reduce churn rate
Champions	Loyalty / referral program	Maximise lifetime value
New customers	Onboarding nurture	Convert to repeat buyers

9 Maturity Scorecard

Criterion	Target
Data Engineering	10
Analytics	10
Machine Learning	9
MLOps	9.5
SDLC Maturity	10
Portfolio Quality	10

Overall ≈ 9.8–10 / 10 for a fresher candidate. With BRD, UML, data modeling, testing, Docker, CI/CD, monitoring, and a business-impact report, the project stops looking like an "AIO exercise" and starts looking like a **mini enterprise data platform**.

Phased Build Order

Phase	Scope	Priority
P0 · Discovery	BRD + Use Case / Activity / Sequence + ERD + data dictionary	MUST
P1 · MVP	Ingest + date-shift → Medallion star schema → RFM Gold → Streamlit DA dashboard → Docker	MUST
P2 · ML + Quality	Churn model + MLflow + batch scoring + dbt/pytest tests + GitHub Actions	SHOULD
P3 · Ops + Impact	K-Means · monitoring (data + model drift) · business impact report · streaming replay	NICE

Retail Customer Intelligence Platform · SDLC-complete overview · AIO Conquer 2026 Module 01